REMEMBER, the agent: a fixed procedure;
the model decides only in steps 3, 4 and 6

- code step; green arrow: made by code
- model step; orange arrow: the model's answer
- dashed arrow: the same data, used again

Input: issue report (evaluated on Jira) and code snapshot

issue tracker · code snapshot

as filed: text, logs, stack trace (if any)

Report (with filed-on date)

Stack Trace

at Files.flush
at Log.write
at Mgr.delete

Logs

report text

the same events (e_1...e_n) with the compressed report

1 Gather: the evidence (code)

from the logs

Logs → e_1 → e_2 …… e_n

printed by

in function

its callers (one hop back)

detection-site files + their callers

from the report text

report + trace → class, function and file names → look up in snapshot → defining files

restated by the model

restate the symptom in code words (the one model call) → behaviour in the vocabulary of the code → content + title search → matching files

from past incidents

incident memory (past issues + their fix files) → firewall: only issues resolved before Filed-On → similar past issues → files their fixes changed

from retrieval

retrieval layer (lexical, no model) → top-20 files

candidate files, each with the reason it was found: "prints event E2", "declares SequenceNumber", "fixed in KAFKA-5342", "retrieval rank 3"

2 Route and candidate table (code)

candidates + reasons

- prose: neither
- logs: matched log line
- trace: stack frames

lead file
trace: frame's target · logs: printing file · prose: past-fix file

candidate table (≤30 rows): file | why	
C1 TransactionManager.java | declares SequenceNumber · past fix KAFKA-5342 · rank 2	always
C2 Sender.java | prints E2 · rank 1	always
C3 ProducerBatch.java | calls the code that prints E2	lead
C4 RecordAccumulator.java | title match	
… sorted by how many clues agree	

Shortlist
first model step

the chosen route

top-2 retrieval files + lead file: always examined

3 Shortlist: which files to open? (model, at most 3 calls)

- report + events e_1...e_n
- candidate table (≤30, with reasons)
- route

the model picks

≤3 calls

picks: ≤5 files to open

code: keeps only paths from the table or its searches; adds the top-2 retrieval files + the lead file

queue: ≤6 files, examined one at a time

≤2 searches, only if the table misses the behaviour; hits join the table

must answer: up to 5 paths + one reason
a prose reply is sent back once, then the step ends

Examine
one file per step

4 Examine: is this file the cause? (model, one file at a time, at most 3 calls each)

fresh context: only these

- one queued file
  whole if ≤24k chars, else its most relevant functions
- compressed report
- every function the file declares

the model judges

may: read more of this file
list a symbol's callers
ask where a value is set

≤3 calls per file

verdict card
role →
≤3 function names, copied from the list
the line the verdict rests on

- **cause: creates the bad state**
- **detects: only reports it**
- **unrelated**

code: drops a function name the file does not contain; at most 8 files per run

verdict cards, one per file

Rank
cards only, never code

5 Walk back (code, once)

≤2 callers of a 'detects' file go back to the queue

6 Rank: which file first? (model, no tools)

cards only, never code

- verdict cards (≤8)
- compressed report
- the snapshot's module list

the model orders

1–2 calls

answer
file order · module (from the list)
chain: symptom → cause, 2–6 steps, each cites Ek or none

code: cause > detects > unrelated, model's order kept inside each group
module and chain steps checked

ranked files + module + chain

Build regions & check
code, no model

7 Build regions and check (code, no model)

from Rank, Examine and Gather

- ranked files + module + chain
- verdict cards (function names)
- retrieval top-20 (from Gather)
- the code snapshot (from Input)

build regions = file + function
card functions first, then 2 keyword picks (marked)
retrieval fills to 10 files

check against what exists
path · function · module
chain step grounded only if it cites or quotes the report

Output: what the agent returns

per issue, all checked

- affected module
- 10 ranked files
  regions = file + function
- causal chain
  each step grounded in a report event